

Total Marks: 7.5

Obtained Marks:

Object Oriented Programming Lab Report

Last Date of Submission: 18th May , 2026

Submitted To: Mr. Muhammad Hassan Mujtaba

Student's Name	Registration Numer
Zymal Khan	25108282
Arsalan Yahya	25108235
Haseeb Manzoor	25108243

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Project Title:

Hospital Management System

Introduction:

The proposed Hospital Management System is developed using the C++ programming language. The system will manage patient information, doctor details, appointments, billing, and hospital records efficiently. This project uses the concepts of Object-Oriented Programming such as classes, objects, inheritance, encapsulation, constructors, and functions.

The system provides an easy interface where hospital staff can add, search, update, and delete records. It reduces paperwork, improves accuracy, saves time, and helps hospital management perform daily operations smoothly.

The system ensures that all information is stored in an organized manner and can be accessed quickly when needed. By applying OOP principles such as encapsulation, abstraction, inheritance, and modularity, the system becomes more structured, reusable, and easy to maintain. This project not only solves real-world problems but also helps students understand practical implementation of OOP concepts. A Hospital Management System is a software application developed to manage the activities and records of a hospital digitally. In traditional hospitals, most information is stored manually in registers and files. This process is time-consuming, difficult to manage, and can lead to errors or data loss. A computerized hospital management system helps hospitals maintain records in a proper and organized manner.

The proposed Hospital Management System is developed using the C++ programming language. The system will manage patient information, doctor details, appointments, billing, and hospital records efficiently. This project uses the concepts of Object-Oriented Programming such as classes, objects, inheritance, encapsulation, constructors, and functions.

The system provides an easy interface where hospital staff can add, search, update, and delete records. It reduces paperwork, improves accuracy, saves time, and helps hospital management perform daily operations smoothly.

2. Background of the Project

Hospitals are one of the most important institutions in society. Every hospital handles hundreds of patients daily. Managing all patient records manually becomes difficult because there are many departments such as reception, pharmacy, billing, emergency, and laboratories.

In many small hospitals and clinics, staff still use paper files for maintaining records. This manual process creates problems such as:

- Difficulty in finding patient records
- Loss of important files
- Time wastage
- Human errors
- Poor record management
- Difficulty in generating bills
- Lack of security

Due to technological advancement, hospitals are shifting toward computerized systems. A Hospital Management System solves these problems by storing data digitally. This project demonstrates how C++ can be used to build a simple yet powerful management system.

3. Problem Statement

Manual hospital management systems create many challenges for hospitals and clinics. Staff members spend a lot of time maintaining files and searching records. Data duplication and human errors are common. Billing processes are also slow and inefficient.

The main problem addressed in this project is:

“To develop a computerized Hospital Management System using C++ that can efficiently manage patient records, doctor details, appointments, and billing information while reducing manual work and improving accuracy.”

4. Purpose of the Project

The purpose of this project is to design and develop a software system that automates hospital operations. The project aims to provide an efficient and user-friendly solution for handling hospital data.

This system helps:

- Store patient records digitally

- Manage doctor information
- Handle appointments
- Generate bills automatically
- Reduce paperwork
- Improve data accuracy
- Save time for hospital staff

The project also helps students understand practical implementation of Object-Oriented Programming concepts in C++.

5. Importance of the Project

This project is important because it demonstrates how software can improve healthcare management. The project helps hospitals maintain records systematically and securely.

The importance of the project includes:

- Improves hospital efficiency
- Saves time for staff and patients
- Reduces paperwork
- Minimizes errors
- Provides quick data access
- Helps in report generation
- Enhances data organization
- Improves patient service quality

For students, this project provides practical experience in programming, problem-solving, and software development.

6. Features of the System

The Hospital Management System contains the following features:

Patient Management

- Add patient details
- View patient records
- Search patient data
- Delete patient records

Doctor Management

- Add doctor information
- Display doctor list
- Manage doctor specialization

Appointment Management

- Schedule appointments
- Store appointment details
- View appointment records

Billing System

- Generate patient bills
- Display treatment charges
- Calculate total payment

User-Friendly Interface

- Menu-driven console system
- Easy navigation
- Clear display of information

7. Functional Requirements

Functional requirements define what the system should do.

Patient Functions

- Add patient record
- Search patient record
- Display all patients
- Delete patient record

Doctor Functions

- Add doctor information
- Display doctor details

Appointment Functions

- Book appointments
- View appointments

Billing Functions

- Enter treatment charges
- Generate bill
- Calculate total amount

8. Non-Functional Requirements

Non-functional requirements define the quality attributes of the system.

Performance

The system should respond quickly to user commands.

Reliability

The system should store and display data correctly.

Usability

The interface should be simple and easy to use.

Maintainability

The code should be understandable and modifiable.

Security

Only authorized users should access sensitive information.

9. System Users

The system can be used by:

Receptionist

- Register patients
- Schedule appointments

Doctors

- View patient details
- Manage treatment information

Administrator

- Manage all records
- Monitor system operations

Billing Staff

- Generate bills
- Handle payment record

10.Project Aims:

The primary aim of this project is to design and implement a reliable and efficient hospital management system using C++ and OOP principles. The system is intended to reduce manual workload and improve accuracy in managing hospital data.

The specific goals include developing a system that can store and manage patient records efficiently, maintain doctor information and availability, and handle billing processes in an organized manner. Another important aim is to ensure that the system is user-friendly so that hospital staff can easily operate it without requiring advanced technical knowledge.

Additionally, the project aims to demonstrate the practical use of OOP concepts such as class design, data hiding, and modular programming. It also focuses on improving problem-solving skills and developing a structured programming approach among students. Concepts like Encapsulation, Inheritance, and Abstraction improve security, reusability, and simplicity.

11.Aims:

- To develop an efficient system using C++ and OOP principles.
- To reduce manual paperwork and human effort.
- To maintain accurate and organized hospital records.
- To improve communication between hospital staff and patients.
- To create a user-friendly system for easy operation.
- To demonstrate practical implementation of OOP concepts.
- To reduce manual work and improve efficiency.
- To apply **Encapsulation** for data security.
- To use **Inheritance** for code re-usability.
- To implement **Abstraction** to hide complex details from users.
- To build a user-friendly and structured system.

12. Objectives of project:

- To store patient details such as name, age, and disease.
- To manage doctor information including specialization and availability.
- To schedule and manage patient appointments.
- To generate bills based on treatment and services.
- To ensure secure and easy data retrieval.
- To improve overall hospital management efficiency
- To store and manage patient and doctor data efficiently.
- To restrict direct data access using encapsulation.
- To reuse common features using inheritance.
- To simplify user interaction using abstraction.
- To provide efficient hospital management functions.

13. Scope of project:

The scope of the Hospital Management System project is quite broad, as it can be used in various healthcare environments such as hospitals, clinics, and small medical centers. The system is designed to handle basic administrative tasks and can be further expanded to include advanced features.

In real-world applications, this system can help hospital staff manage large amounts of data efficiently. It reduces the chances of data loss and improves coordination between different departments. The project can also be upgraded in the future by integrating databases, graphical user interfaces (GUI), and online services.

This makes the project highly flexible and scalable, allowing it to adapt to the needs of different healthcare institutions.

- Can be used in hospitals, clinics, and medical centers.
- Helps in managing large amounts of data efficiently.
- Reduces chances of data loss and mismanagement.
- Can be expanded with advanced features in the future.
- Suitable for both small-scale and large-scale healthcare systems.

Step-by-Step Implementation Process:

- **Requirement Analysis:** Identify system needs (patients, doctors, billing).
- **System Design:** Create classes like Patient, Doctor, Hospital.
- **Coding:** Implement using C++ with OOP concepts (classes, objects, encapsulation).
- **Testing:** Check system for errors and fix bugs.
- **Execution:** Run the system and evaluate performance.

Detailed Implementation Using Classes, Objects, and Core OOP Concepts in C++ :

The Hospital Management System is implemented using Object-Oriented Programming concepts in C++. The system is divided into multiple classes to ensure proper organization and modularity.

The Patient class is responsible for storing patient details such as name, age, and disease. The Doctor class manages doctor information, including their name, specialization, and availability. The Hospital class acts as the main controller that manages patients, doctors, and overall system operations.

Encapsulation is achieved by keeping data members private and accessing them through public

member functions. Functions are used to add, display, and update records. This structure ensures that the system is secure, maintainable, and easy to understand.

14. Explanation of System Working and User Interaction :

The system works as a menu-driven program, allowing users to interact with it easily. When the program starts, a menu is displayed with different options such as adding a patient, viewing patient details, assigning a doctor, and generating a bill.

The user selects an option by entering a number, and the system performs the corresponding operation. For example, when a user selects the option to add a patient, the system asks for input details and stores them in the program.

Similarly, when generating a bill, the system calculates and displays the total cost based on the services provided. This interactive approach makes the system user-friendly and easy to operate even for beginners.

15. Explanation of System Working and User Interaction :

The system works as a menu-driven program, allowing users to interact with it easily. When the program starts, a menu is displayed with different options such as adding a patient, viewing patient details, assigning a doctor, and generating a bill.

The user selects an option by entering a number, and the system performs the corresponding operation. For example, when a user selects the option to add a patient, the system asks for input details and stores them in the program.

Similarly, when generating a bill, the system calculates and displays the total cost based on the services provided. This interactive approach makes the system user-friendly and easy to operate even for beginners.

16. Advantages of project :

- Reduces manual paperwork and saves time for hospital staff.
- Minimizes human errors in record management.
- Provides fast and easy access to patient and doctor information.
- Ensures **data security using encapsulation** (private data protection).
- Promotes **code reusability through inheritance**, reducing redundancy.
- Simplifies system usage through **abstraction**, hiding complex logic.
- Improves communication between departments (doctors, reception, billing).
- Makes record management more organized and systematic.
- Easy to update and maintain due to modular structure.
- Enhances overall hospital efficiency and productivity.
- Allows multiple operations (add, update, delete, view data).
- Reduces chances of data duplication and inconsistency.
- Provides a structured and logical approach to problem-solving.
- Useful for learning real-world implementation of OOP concepts.
- Can be scaled and extended for larger systems.

17. Limitations of the project :

- Data is **not stored permanently** (no database integration).
- System is **console-based**, lacking graphical user interface (GUI).
- Cannot handle **large-scale hospital operations efficiently**.
- Limited features (no pharmacy, lab, or emergency modules).
- No **online access or remote connectivity**.
- Data security is basic (no authentication or login system).
- Cannot handle multiple users at the same time.
- No backup or recovery system for data loss.
- Performance may decrease with large data input.
- Requires basic technical knowledge to operate.
- No real-time updates or notifications.
- Limited validation of user input (may cause errors).
- Does not support integration with other systems (e.g., lab systems).
- Lacks reporting and analytics features.

18. Future Enhancements :

- Integrate **database (MySQL/SQL)** for permanent data storage.
- Develop a **Graphical User Interface (GUI)** for better usability.
- Add **user authentication system (login/signup)** for security.
- Implement **online appointment booking system**.
- Add **pharmacy management module** for medicine records.
- Include **laboratory management system** for test reports.
- Enable **multi-user access** for different hospital staff roles.
- Add **report generation and data analytics** features.
- Integrate **cloud storage** for remote data access.
- Implement **mobile application support**.
- Add **notification system** (SMS/Email alerts).
- Improve **data validation and error handling mechanisms**.
- Include **backup and recovery system** for data safety.
- Use **file handling or database encryption** for better security.
- Expand system for **large hospitals and healthcare networks**.
- Apply advanced OOP concepts like **polymorphism and interfaces**.

19. System Working Procedure

The working procedure of the system is as follows:

1. The user opens the program.
2. The main menu appears on the screen.
3. The user selects an option.
4. The system performs the selected operation.
5. Data is stored and displayed.
6. The user may continue using other options.
7. The program exits when the user selects Exit.

15. Object-Oriented Programming Concepts Used

This project uses important OOP concepts.

Classes and Objects

Classes are used to define hospital entities such as Patient and Doctor. Objects are created from these classes.

Encapsulation

Data members are kept private and accessed using public member functions.

Constructors

Constructors initialize object data automatically.

Inheritance

One class can inherit features from another class.

Conclusion:

In conclusion, the Hospital Management System developed using Object-Oriented Programming (OOP) concepts provides an efficient and organized solution for managing hospital operations. By applying key principles such as encapsulation, inheritance, and abstraction, the system ensures data security, code reusability, and simplified user interaction. It reduces manual work, minimizes errors, and improves the overall efficiency of handling patient and hospital records. Although the current system has some limitations, it serves as a strong foundation for building more advanced healthcare applications. This project not only demonstrates the practical implementation of OOP concepts but also highlights how technology can play a vital role in improving modern healthcare systems.

C++ Code:

```
#include <iostream>
#include <string>
using namespace std;

// Patient Class
class Patient
{
private:
```

Shaheed Zulfikar Ali Bhutto Institute of Science & Technology University

```
int patientID;
string name;
int age;
string disease;

public:
void addPatient()
{
    cout << "\nEnter Patient ID: ";
    cin >> patientID;

    cin.ignore();

    cout << "Enter Patient Name: ";
    getline(cin, name);

    cout << "Enter Patient Age: ";
    cin >> age;

    cin.ignore();

    cout << "Enter Disease: ";
    getline(cin, disease);
}

void displayPatient()
{
    cout << "\n----- Patient Record -----" << endl;
    cout << "Patient ID: " << patientID << endl;
    cout << "Name: " << name << endl;
    cout << "Age: " << age << endl;
    cout << "Disease: " << disease << endl;
}
};

// Doctor Class
class Doctor
{
private:
    int doctorID;
    string doctorName;
    string specialization;

public:
    void addDoctor()
    {
        cout << "\nEnter Doctor ID: ";
        cin >> doctorID;

        cin.ignore();
```

Shaheed Zulfikar Ali Bhutto Institute of Science & Technology University

```
cout << "Enter Doctor Name: ";
getline(cin, doctorName);

cout << "Enter Specialization: ";
getline(cin, specialization);
}

void displayDoctor()
{
    cout << "\n----- Doctor Record -----" << endl;
    cout << "Doctor ID: " << doctorID << endl;
    cout << "Doctor Name: " << doctorName << endl;
    cout << "Specialization: " << specialization << endl;
}
};

// Billing Class
class Billing
{
private:
    float charges;

public:
    void generateBill()
    {
        cout << "\nEnter Treatment Charges: ";
        cin >> charges;

        cout << "Total Bill: " << charges << " PKR" << endl;
    }
};

// Main Function
int main()
{
    Patient patients[100];
    Doctor doctors[50];
    Billing bill;

    int patientCount = 0;
    int doctorCount = 0;

    int choice;

    do
    {
        cout << " HOSPITAL MANAGEMENT SYSTEM" << endl;
        cout << "1. Add Patient" << endl;
        cout << "2. Display Patients" << endl;
        cout << "3. Add Doctor" << endl;
```



```
cout << "4. Display Doctors" << endl;
cout << "5. Generate Bill" << endl;
cout << "6. Exit" << endl;

cout << "\nEnter Your Choice: ";
cin >> choice;

switch(choice)
{
case 1:
    patients[patientCount].addPatient();
    patientCount++;
    cout << "\nPatient Added Successfully!" << endl;
    break;

case 2:
    if(patientCount == 0)
    {
        cout << "\nNo Patient Records Found!" << endl;
    }
    else
    {
        for(int i = 0; i < patientCount; i++)
        {
            patients[i].displayPatient();
        }
    }
    break;

case 3:
    doctors[doctorCount].addDoctor();
    doctorCount++;
    cout << "\nDoctor Added Successfully!" << endl;
    break;

case 4:
    if(doctorCount == 0)
    {
        cout << "\nNo Doctor Records Found!" << endl;
    }
    else
    {
        for(int i = 0; i < doctorCount; i++)
        {
            doctors[i].displayDoctor();
        }
    }
    break;

case 5:
```

```
bill.generateBill();
    break;

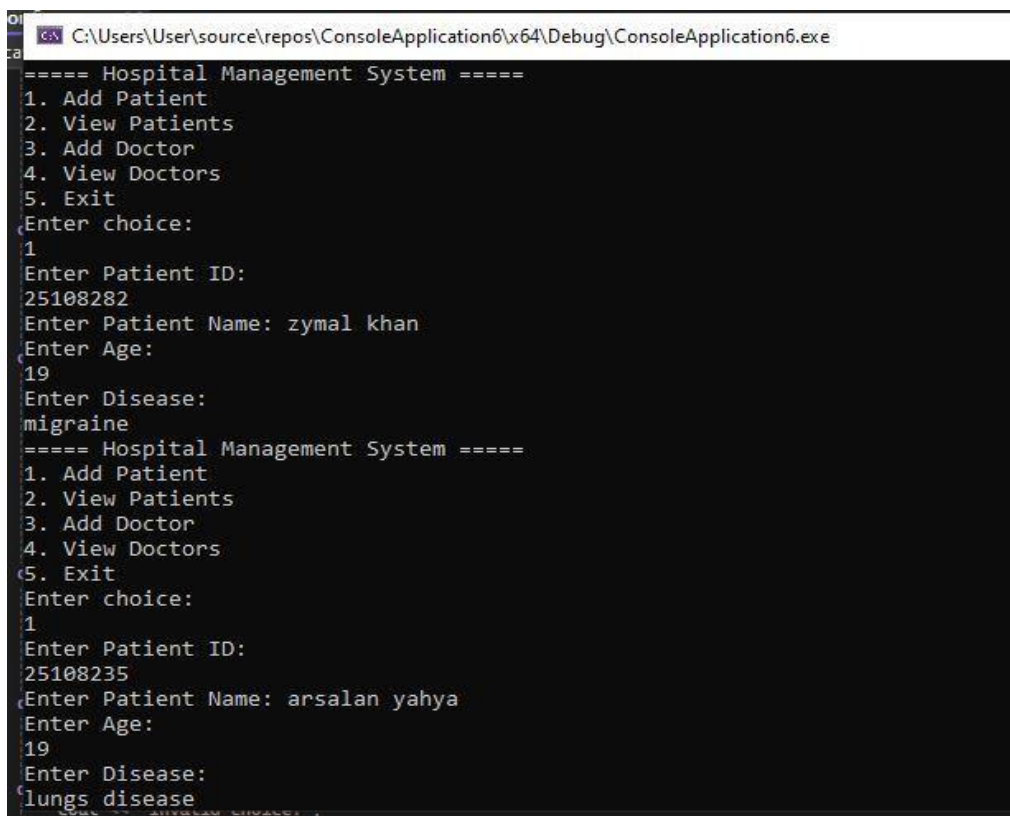
    case 6:
        cout << "\nExiting Program..." << endl;
        break;

    default:
        cout << "\nInvalid Choice!" << endl;
    }

} while(choice != 6);

return 0;
}
```

OUTPUT:



```
C:\Users\User\source\repos\ConsoleApplication6\x64\Debug\ConsoleApplication6.exe
===== Hospital Management System =====
1. Add Patient
2. View Patients
3. Add Doctor
4. View Doctors
5. Exit
Enter choice:
1
Enter Patient ID:
25108282
Enter Patient Name: zymal khan
Enter Age:
19
Enter Disease:
migraine
===== Hospital Management System =====
1. Add Patient
2. View Patients
3. Add Doctor
4. View Doctors
5. Exit
Enter choice:
1
Enter Patient ID:
25108235
Enter Patient Name: arsalan yahya
Enter Age:
19
Enter Disease:
lungs disease
```

```
==== Hospital Management System ====
1. Add Patient
2. View Patients
3. Add Doctor
4. View Doctors
5. Exit
Enter choice:
2
Patient ID: 25108282
Name: zymal khan
Age: 19
Disease: migraine
Patient ID: 25108235
Name: arsalan yahya
Age: 19
Disease: lungs disease
Patient ID: 25108243
Name: haseeb manzoor
Age: 19
Disease: dimaag kharab
```

